

# Reviewer Pack — Laplacian Apolar Rank Caps Delorme-Poljak Max-CUT SoS-2 Gap

Entry 378d9a4dead5 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 00:24:05 UTC

## Laplacian Apolar Rank Caps Delorme-Poljak Max-CUT SoS-2 Gap

Entry ID: 378d9a4dead5 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 00:24:05 UTC

### 1. Conjecture

**Field A** (mathematical branch): Curto-Fialkow real moment problem / apolarity & Hankel flat-extension theory (real algebraic geometry). The Hankel rank of the Laplacian moment sequence  $m_k(G) := \text{tr}(L_G^k)$  is by the Hamburger truncated moment theorem the smallest support size of an atomic measure on  $\mathbb{R}_{\geq 0}$  reproducing the moments — equivalently the number of distinct Laplacian eigenvalues (Curto-Fialkow 1996; Lasserre 2009 ‘Moments, Positive Polynomials and their Applications’; Blekherman-Parrilo-Thomas 2012 ‘Semidefinite Optimization and Convex Algebraic Geometry’; Reznick 1992). The invariant uses real-eigenvalue clustering and rank-of-Hankel — non-ring (no  $\mathbb{F}_q$  polynomial extension preserves cluster count of real eigenvalues under polynomial ring lifts), Aaronson-Wigderson algebrization-safe. An arXiv search ‘Curto-Fialkow flat extension’ AND (‘Max-Cut’ OR ‘Delorme-Poljak’ OR ‘SoS integrality gap’) returns 0 direct hits with <5 adjacent papers (Lasserre hierarchy convergence proofs, Helton-Nie semidefinite representability), none using Hankel-apolar rank of the Laplacian-moment sequence as an SoS-2 Max-CUT tightness invariant. Distinct from blacklisted Lee-Yang zeros of the cut polynomial (complex roots, not real moment-sequence), Fekete-capacity (transfinite diameter of spectrum), free 4-cumulant excess (Voiculescu NC partition statistics), Kashin  $L^1$ -flatness (single-eigenvector  $L^{1/L_2}$  ratio), and Halász  $L^2$  discrepancy (CDF deviation). **Field B** (complexity object): Degree-2 Sum-of-Squares / Delorme-Poljak integrality gap  $\rho(G) := n \cdot \lambda_{\max}(L_G) / (4 \cdot \text{MC}(G)) - 1$  for the Max-CUT problem on connected  $k$ -regular graphs (Goemans-Williamson 1995; Delorme-Poljak 1993; Schoenebeck 2008; Barak-Steurer 2014). The structural form of the Hankel-rank invariant (a count of distinct Laplacian eigenvalues, a property of the GRAPH WIRING — two non-isomorphic graphs with identical Max-CUT value can have very different  $h(G)$ ) shields it from the Razborov-Rudich natural-proofs barrier; using only real-eigenvalue grouping and integer counting it lies outside the algebrization closure of Aaronson-Wigderson.

#### Statement:

For every connected  $k$ -regular simple graph  $G$  on  $n \in [8, 20]$  vertices with  $k \in \{3, 4\}$ , let  $h(G)$  be the number of distinct nonzero Laplacian eigenvalues of  $L_G$  (equivalently the rank of the truncated Hankel matrix  $H_d(G)_{\{i,j\}} = \text{tr}(L_G^{i+j+1})$  for  $d \geq n$ , the Curto-Fialkow apolar rank of the Laplacian moment sequence). Let  $\text{MC}(G)$  be the exact Max-CUT value and  $\rho(G) := n \cdot \lambda_{\max}(L_G) / (4 \cdot \text{MC}(G)) - 1$  be the Delorme-Poljak SoS-2 integrality gap. Then  $n \cdot \rho(G) \leq h(G) \cdot \log_2(n+1)$ ; a single instance with  $n \cdot \rho(G) > 2 \cdot h(G) \cdot \log_2(n+1)$  refutes the conjecture.

#### Rationale (proposer’s reasoning):

Apolarity / Curto-Fialkow theory identifies the Hankel rank of a moment sequence with the minimum number of atoms in a representing measure — a canonical real-algebraic-geometry obstruction. Graphs whose Laplacian spectra collapse onto few atoms (distance-regular, SRG, complete bipartite) tend to admit tight Delorme-Poljak SoS-2 certificates because the symmetry forces the optimal pseudoexpectation onto a low-dimensional invariant subspace. The conjecture promotes the folk ‘symmetric spectra  $\Rightarrow$  tight SoS-2’ heuristic to a quantitative apolar bound, suggesting Hankel rank as the moment-matrix property P sought by the SOS\_DEGREE focus.

**Taxonomy category:** GCT (status at proposal time: alive)

## 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** 082452b538fc4ab4

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

**Acceptance criterion:**

Across 240 random connected  $k$ -regular instances (30 per  $(n,k)$  in the 8 specified cells), compute  $r = n \cdot \rho(G) / (h(G) \cdot \log_2(n+1))$ . Conjecture is SUPPORTED iff  $\max r$  over all seeds  $< 1$  (i.e.  $n \cdot \rho \leq h \cdot \log_2(n+1)$  holds for every instance) AND mean  $r \leq 0.75$ . FALSIFIED iff any single instance yields  $r \geq 2$ .

## 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|---------|------------|------------------|------------------|
| RELATIVIZATION | SAFE    | 0.96       | SAFE             | SAFE             |
| NATURAL PROOFS | SAFE    | 0.90       | SAFE             | SAFE             |
| ALGEBRIZATION  | SAFE    | 0.88       | SAFE             | SAFE             |
| KARP_LIPTON    | SAFE    | 0.97       | SAFE             | SAFE             |

## 4. Novelty audit

**Verdict:** NOVEL against 7 hits across arXiv + Semantic Scholar + ECCC

**Search queries (3):** - Laplacian distinct eigenvalues Max-Cut Delorme-Poljak integrality gap - Hankel moment rank Laplacian spectrum sum-of-squares Max-Cut - Curto-Fialkow flat extension Laplacian eigenvalues SoS hierarchy Max-Cut

**Top relevant hits considered:** - [<http://arxiv.org/abs/1107.1630v3>] On the Integrality Gap of the Subtour LP for the 1,2-TSP - [<http://arxiv.org/abs/2404.18142v1>] Revisiting Majumdar-Ghosh spin chain model and Max-cut problem using variational quantum algorithms - [<http://arxiv.org/abs/2411.11721v4>] Eigenvalues of the Neumann magnetic Laplacian in the unit disk - [<http://arxiv.org/abs/0812.4567v1>] On degenerate Hamburger moment problem and extensions of positive semidefinite Hankel block matrices - [<http://arxiv.org/abs/1508.00617v2>] Hankel determinants of random moment sequences - [<http://arxiv.org/abs/2108.06635v1>] Sums of squares, Hankel index, and almost real rank - [<http://arxiv.org/abs/1304.3212v2>] Eigenvalues of the Witten-Laplacian on compact Riemannian manifolds

## 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only,  $\leq 90$ s wall time per cycle **Execution:** rc=1, elapsed=0.1s

## 5.1 Generated Python source

```
import random
import math
from collections import defaultdict

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        # Find pivot row
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]

        # Eliminate below
        for j in range(i+1, n):
            factor = A[j][i] / A[i][i]
            for k in range(n):
                A[j][k] -= factor * A[i][k]

    # Back-substitute to find solution
    x = [0] * n
    for i in range(n-1, -1, -1):
        x[i] = A[i][-1] / A[i][i]
        for j in range(i-1, -1, -1):
            A[j][-1] -= A[j][i] * x[i]

    return x

def matrix_multiply(A, B):
    n = len(A)
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def laplacian_eigenvalues(G):
    n = len(G)
    D = [[0] * n for _ in range(n)]
    for i in range(n):
        degree = sum(G[i])
        D[i][i] = degree
    L = matrix_multiply(D, G)
    eigenvalues = gaussian_elimination(L)
    return sorted(eigenvalues, reverse=True)

def max_cut(graph):
    n = len(graph)
    best_cut_value = 0

    for i in range(1 << (n-1)):
        cut_value = 0
        for j in range(n):
```

```

        for k in range(j+1, n):
            if (i >> j) & 1 and (i >> k) & 1:
                cut_value += graph[j][k]
            elif not ((i >> j) & 1) and not ((i >> k) & 1):
                cut_value -= graph[j][k]
        best_cut_value = max(best_cut_value, abs(cut_value))

    return best_cut_value

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [8, 10, 12, 14, 16]
    k_values = [3, 4]
    results = []

    for n in n_values:
        for k in k_values:
            graph = [[0] * n for _ in range(n)]
            degree_sum = 0
            while degree_sum != n * k:
                u = random.randint(0, n-1)
                v = random.randint(0, n-1)
                if u == v or graph[u][v] == 1:
                    continue
                graph[u][v] = 1
                graph[v][u] = 1
                degree_sum += 2

            eigenvalues = laplacian_eigenvalues(graph)
            h_G = len([e for e in eigenvalues if e != 0])
            lambda_max = max(eigenvalues)
            MC_G = max_cut(graph)
            rho_G = n * lambda_max / (4 * MC_G) - 1
            r = n * rho_G / (h_G * math.log2(n+1))

            results.append(r)

    metric_value = max(results)
    conjecture_holds = metric_value < 2
    counterexample = "" if conjecture_holds else f"max_r={metric_value}"

    return {
        "metric_name": "n·p(G)/(h(G)·log2(n+1))",
        "metric_value": metric_value,
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    results = []
    for seed in seeds:
        trial = run_trial(seed)

```

```

print(f"TRIAL: {trial}")
results.append(trial["metric_value"])

mean_value = sum(results) / len(results)
support_fraction = sum(1 for r in results if r < 2) / len(results)

if all(r < 2 for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
elif any(r >= 2 for r in results):
    first_failing_seed = next(seed for seed, trial in enumerate(results) if trial["metric_value"] >= 2)
    print(f"RESULT: FALSIFIED counterexample=\"max_r={results[first_failing_seed]}\\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE reason=unknown")

```

## 6. Per-seed results

*(no seed-level data — test crashed before producing TRIAL: lines)*

## 7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_06da5d96.py", line 123, in <module>
    trial = run_trial(seed)
            ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_06da5d96.py", line 96, in run_trial
    eigenvalues = laplacian_eigenvalues(graph)
                  ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_06da5d96.py", line 58, in laplacian_eigenvalues
    eigenvalues = gaussian_elimination(L)
                  ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_06da5d96.py", line 29, in gaussian_elimination
    factor = A[j][i] / A[i][i]
              ~~~~~^~~~~~
ZeroDivisionError: float division by zero

```

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

The test crashed with a ZeroDivisionError in the custom gaussian\_elimination routine before any trials completed, so no r-values were produced and neither the support nor falsification condition can be evaluated. | next: Replace the hand-rolled Gaussian elimination with numpy.linalg.eigvalsh (or use pivoting) to robustly compute Laplacian eigenvalues, then rerun the 240-seed sweep.

## 11. Audit log (LLM calls)

Total LLM calls: 9

| # | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|---|-----------------|---------------|------------------|-----------|-----|--------------|
| 1 | propose         | claude_max    | opus             | 0         | 0   | 332863       |
| 2 | preregistration | claude_max    | opus             | 0         | 0   | 5002         |
| 3 | novelty         | claude_max    | opus             | 0         | 0   | 10622        |
| 4 | novelty         | claude_max    | opus             | 0         | 0   | 9721         |
| 5 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 15914        |
| 6 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 13698        |
| 7 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 12411        |
| 8 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 13709        |
| 9 | judge           | claude_max    | opus             | 0         | 0   | 4300         |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 418240 ms total latency. Provider mix: {'claude\_max': 5, 'ollama\_remote': 4}

(full prompt+response transcripts available in `research/audit/378d9a4dead5.jsonl`)

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/378d9a4dead5.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** `research/pvsnp_sandbox/test_*.py` (per-cycle)

**Replay tarball:** `research/replay/378d9a4dead5.tar.gz` (if generated)